

Steam 游戏商店大数据分析报告

课程：大数据应用 · 期末项目
团队：罗马 (Roma)
成员：罗景楠、马亦麟
代码仓库：<https://github.com/yilinpotato/BigData-App>

0. 最终提交物与评分重点

最终建议提交以下材料：

材料	具体内容	作用
项目仓库	GitHub 仓库： https://github.com/yilinpotato/BigData-App	包含完整代码、脚本、SQL、Notebook、图表、报告与仪表盘文件
最终报告	report/report_en.pdf	英文正式报告，对应报告部分评分
可执行分析	notebooks/steam_analysis.ipynb、src/*.py	证明 Spark/HDFS 清洗与分析流程可复现
SQL / HiveQL	sql/analysis_queries.sql、sql/hive_setup.hql	证明结构化查询与 Hive 数仓实现
可视化结果	figures/*.png 与 Dashboard 截图	支撑报告中的关键发现
Dashboard 加分项	https://bigdata-app-nckiksi6ebas3bxbgdr5m9.streamlit.app/	交互式展示，争取扩展实现加分

评分主要分为两部分：

- **项目报告 (50 分)**：数据集介绍、HDFS 存储/访问策略、数据预处理、分析问题与方法、关键发现、可视化解释、结论反思、整体连贯性、展示质量和团队分工。
- **源代码 (50 分)**：Notebook 或脚本是否完整可执行，Spark/HDFS 加载与清洗管线是否合理，工具使用是否正确，代码结构与注释质量，SQL/HiveQL 是否完整，方法是否有深度，结果和演示是否完整。
- **可选加分 (最多 10 分)**：Dashboard 或 Web 界面等扩展实现。本项目已提供包含 8 个交互式分析 Tab 的 Streamlit Dashboard。

1. 引言与数据集介绍

电子游戏是全球最大的数字娱乐产业之一，Steam 是 PC 平台最主要的游戏分发渠道。本项目对 Steam 商店约 2.7 万款游戏的数据进行大数据分析，挖掘市场结构、定价规律、用户粘性、平台战略等方面的规律，完整演练 **HDFS 存储 -> PySpark 清洗 -> Spark SQL / HiveQL 分析 -> 可视化 -> 交互式仪表盘展示** 的大数据处理流程。项目最终交付包括可执行脚本、已运行的 Notebook、SQL/HiveQL 查询、8 张可视化图表、PDF 报告以及 Streamlit Dashboard。

数据集来自 Kaggle [Steam Store Games \(Clean Dataset\)](#)，由 SteamSpy 与 Steam Storefront API 抓取 (2019 年快照)，共 **6 个 CSV**、约 **242 MB**，以 `appid` 为主键关联：

文件	行数	内容
steam.csv	27,075	主表：名称、发行日期、开发/发行商、平台、类型、好评/差评数、游玩时长、拥有量、价格
steamspy_tag_data.csv	29,022	标签宽表（约 370 个标签列，值为投票数）
steam_description_data.csv	174,903	游戏描述文本
steam_media_data.csv	27,332	图片/截图/视频链接（嵌套 JSON）
steam_requirements_data.csv	27,319	配置要求（含 HTML）
steam_support_info.csv	27,136	官网/客服链接

本报告的核心分析基于 steam.csv（主表）与 steamspy_tag_data.csv（标签）。

2. 系统架构与 HDFS 存储 / 访问策略

2.1 技术栈

层	组件	版本
存储	Hadoop HDFS（单机伪分布式）	3.3.6
计算	Apache Spark / PySpark	3.5.5
查询	Spark SQL	3.5.5
数仓	Hive Metastore / HiveQL（Spark Hive Support）	与 Spark 3.5.5 集成
运行时	OpenJDK	17 (Temurin)
可视化	matplotlib / seaborn	—
展示	Streamlit / Altair	Dashboard 加分项

整个栈在 **WSL2 + Ubuntu 22.04** 上以用户级、无需 root 的方式部署。

2.2 HDFS 存储策略

采用「原始区 + 清洗区」分层的数据湖式组织：

hdfs:///steam/raw/	原始 CSV（不可变，留档）
hdfs:///steam/clean/	清洗后的列式数据
├─ games/	主表，Parquet，按 release_year 分区
└─ tags_long/	标签长表，Parquet

设计要点：

- **原始/清洗分离**：原始 CSV 上传到 /steam/raw 后保持只读，所有加工产物写入 /steam/clean，保证可追溯、可重跑。
- **列式 + 压缩（Parquet）**：清洗后由行式 CSV 转为 **Parquet**，列式存储 + 内置压缩，显著降低分析时的 I/O，并保留 schema 与类型。
- **分区（Partitioning）**：主表按 release_year 分区，使「按年份」的查询能分区裁剪（**partition pruning**），只扫描相关年份目录。
- **副本数=1**：单节点环境设 dfs.replication=1，避免无谓冗余。

2.3 访问方式与工程难点

- HDFS 守护进程以 `hdfs --daemon start namenode/datanode` 直接启动，**绕过 start-dfs.sh 对 SSH 免密登录的依赖**（WSL 下无 sshd），实现无 root 单机部署。

- Spark 通过 `spark.read.parquet("hdfs://localhost:9000/steam/clean/games")` 直接读 HDFS。
- 工程中解决了三个真实问题，并固化到脚本：① **Java 17 与 Hadoop 的模块访问限制**——为守护进程注入 `--add-opens`；② **工作目录含空格导致 Hadoop FsShell 的本地路径 URI 解析失败**——用硬链接把数据暴露到无空格目录再 `put`；③ Spark 本地落盘目录重定向到无空格路径。

全部启动脚本见仓库 `setup/` (`00_install_stack.sh` / `01_start_hdfs.sh` / `02_load_data.sh` / `env.sh`)。

2.4 Hive 数仓与交互式展示

在 Spark 清洗层之上，项目额外实现了 Hive 数据仓库与仪表盘两个扩展模块：

- **Hive 数仓**：`src/hive_warehouse.py` 启用 `enableHiveSupport()`，将清洗后的 Parquet 物化为 `hdfs:///steam/warehouse/games`，并在 `steam` 数据库下建立 `games` 与 `tags_long` 两张外部表。`sql/hive_setup.hql` 给出建库、建表以及发行趋势、价格口碑、热门标签三个 HiveQL 查询，证明数据可以通过 SQL 数仓方式复用。
- **Streamlit Dashboard**：`src/export_dashboard_data.py` 用 Spark 预计算 8 个分析问题的聚合结果，导出到 `app/data/*.csv`；`app/dashboard.py` 在运行时只依赖 `pandas`、`Streamlit` 与 `Altair`，因此可以脱离 HDFS/Spark 部署。仪表盘包含概览指标卡与 8 个交互式分析 Tab，作为代码部分的扩展实现与演示加分项。

2.5 Dashboard 可视化展示

已部署的 Dashboard 将 Spark 计算结果做成交互式页面，包含 5 个概览指标，以及与 8 个分析问题——对应的 Tab。



价格 Tab 使用双轴交互图：柱状图表示各价格区间平均好评率，折线表示估计平均拥有量，使“低价不等于走量”的结论在演示时更直观。

团队 罗马 (Roma)

成员：罗景楠·马亦麟

课程：大数据应用 期末项目

技术栈

HDFS · PySpark · Spark SQL · Streamlit

[GitHub 仓库](#)

数据：Kaggle Steam Store Games (约 2.7 万款游戏)

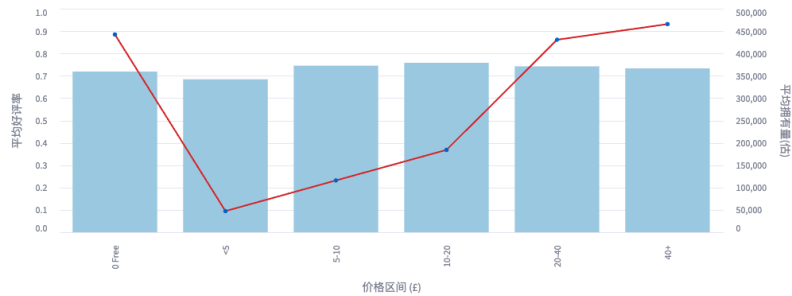
Steam 游戏商店大数据分析

基于 HDFS + Spark 处理约 2.7 万款 Steam 游戏，交互展示 8 个分析维度。

游戏总数 27,075 免费 / 付费 2,560 / 24... 平均好评率 71.5% 中位价格 £3.99 Mac / Linux 支持 30% / 19%

Q1 发行趋势 Q2 类型 Q3 价格 Q4 时长 Q5 厂商 Q6 平台 Q7 标签 Q8 成就

价格区间 × 好评率 / 拥有量



最低价区(<£5)好评率最低(0.684)、拥有量也最低；中端(£10-40)口碑最佳；免费与精品拥有量最大——低价≠走量。

© 团队罗马·数据 2019 快照·由 Spark/HDFS 预计算，Streamlit 呈现

Created by yilinpoto

Hosted with Streamlit

3. 数据预处理 (Spark 清洗管线)

清洗管线 `src/clean_pipeline.py` 从 HDFS 读原始 CSV，输出清洗后的 Parquet，关键步骤：

- 类型转换**：`release_date` -> `date`、`评价数/时长` -> 整型、`price` -> `double`、布尔字段转换。
- 派生字段**：
 - `positive_ratio = positive / (positive + negative)` (好评率)
 - `total_ratings`、`is_free` (价格为 0)
 - `owners_low` / `owners_high` / `owners_mid`：将拥有量区间字符串 (如 `10000000-20000000`) 解析为上下界与中位估计
 - `num_genres` / `num_categories` / `num_platforms` 及 `on_windows` / `on_mac` / `on_linux` 平台布尔
- 多值字段处理**：`platforms` / `categories` / `genres` / `steamspy_tags` 以 ; 分隔，分析时用 `explode` 展开。
- 缺失值与异常处理**：过滤无 `appid` 或空 `name` 的记录、`appid` 去重、负价归零。
- 宽表转长表**：标签宽表 (约 370 列) 用 `Spark stack()` 转为长表 (`appid, tag, votes`) 并过滤 `votes > 0`，由约 1,000 万稀疏单元压缩为 **215,633** 条有效记录。

清洗结果：主表 **27,075** 行 (全部有效)，整体平均好评率 **0.714**，免费游戏 **2,560** 款。

3.1 关键清洗代码

下面的代码片段展示了清洗管线如何把原始字符串字段转换为可分析字段，并派生评价、价格、拥有量和平台指标：

```

g = (raw
    .withColumn("appid", F.col("appid").cast(T.IntegerType()))
    .withColumn("release_date", F.to_date("release_date", "yyyy-MM-dd"))
    .withColumn("price", F.col("price").cast(T.DoubleType()))
    .withColumn("release_year", F.year("release_date"))
    .withColumn("total_ratings", F.col("positive_ratings") + F.col("negative_ratings"))
    .withColumn(
        "positive_ratio",
        F.when(F.col("total_ratings") > 0,
            F.col("positive_ratings") / F.col("total_ratings"))
    )
    .withColumn("is_free", F.col("price") == 0)
    .withColumn("owners_low", F.split("owners", "-").getItem(0).cast(T.LongType()))
    .withColumn("owners_high", F.split("owners", "-").getItem(1).cast(T.LongType()))
    .withColumn("owners_mid", ((F.col("owners_low") + F.col("owners_high")) / 2).cast(T.LongType()))
    .withColumn("on_windows", F.col("platforms").contains("windows"))
    .withColumn("on_mac", F.col("platforms").contains("mac"))
    .withColumn("on_linux", F.col("platforms").contains("linux")))

```

标签表原本是稀疏宽表，使用 Spark `stack()` 转为长表，便于后续 SQL 聚合：

```

tag_cols = [c for c in tags.columns if c != "appid"]
pairs = ", ".join([f"'{c}', '{c}'" for c in tag_cols])
stack_expr = f"stack({len(tag_cols)}, {pairs}) as (tag, votes)"
tags_long = (tags.select(F.col("appid").cast(T.IntegerType()), F.expr(stack_expr))
    .withColumn("votes", F.col("votes").cast(T.IntegerType()))
    .filter(F.col("votes") > 0))

```

4. 分析问题、方法与发现

围绕 8 个分析问题，项目统一以 **Spark SQL / DataFrame API** 计算聚合结果，再用 matplotlib/seaborn 可视化；其中核心查询整理为 `sql/analysis_queries.sql`，部分查询也通过 Hive 外部表在 `sql/hive_setup.hql` 中复现。完整分析流程见已执行的 `notebooks/steam_analysis.ipynb` 与导出的 `src/steam_analysis.py`。

4.1 关键查询与可视化代码

价格分析使用 Spark SQL 构造价格分桶；同一聚合结果导出到 Dashboard 后，用 Altair 做双轴交互图：

```

SELECT CASE
    WHEN price = 0 THEN '0 Free'
    WHEN price < 5 THEN '<5'
    WHEN price < 10 THEN '5-10'
    WHEN price < 20 THEN '10-20'
    WHEN price < 40 THEN '20-40'
    ELSE '40+' END AS price_band,
COUNT(*) AS n,
ROUND(AVG(positive_ratio), 3) AS avg_ratio,
ROUND(AVG(owners_mid), 0) AS avg_owners
FROM games
GROUP BY price_band;

```

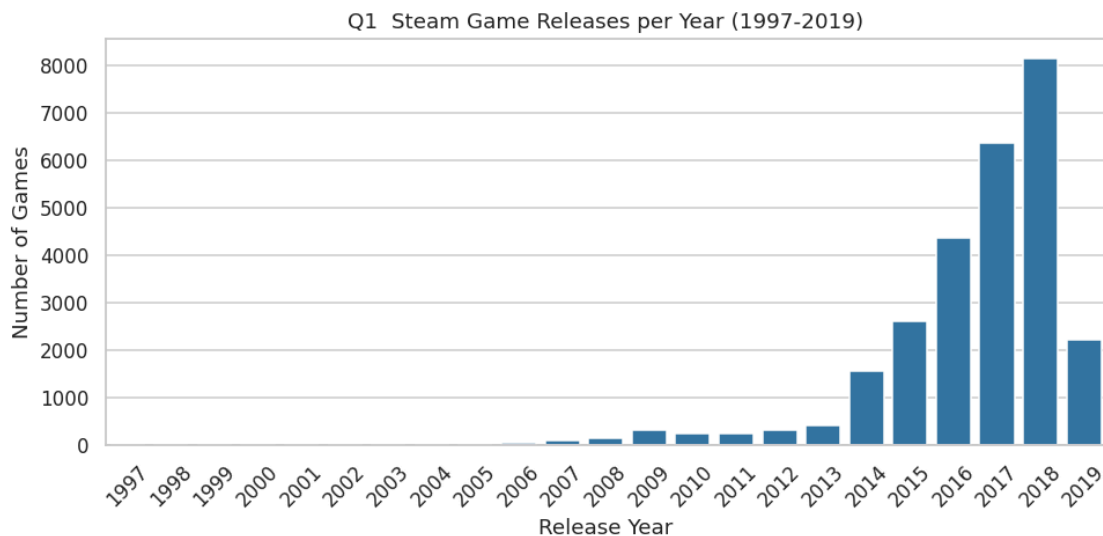
```

base = alt.Chart(d).encode(x=alt.X("price_band:N", sort=order))
bar = base.mark_bar(color="#9ecae1").encode(y="avg_ratio:Q")
line = base.mark_line(point=True, color="#d62728").encode(y="avg_owners:Q")
st.altair_chart(alt.layer(bar, line).resolve_scale(y="independent"),
    width="stretch")

```

Q1 年度发行趋势

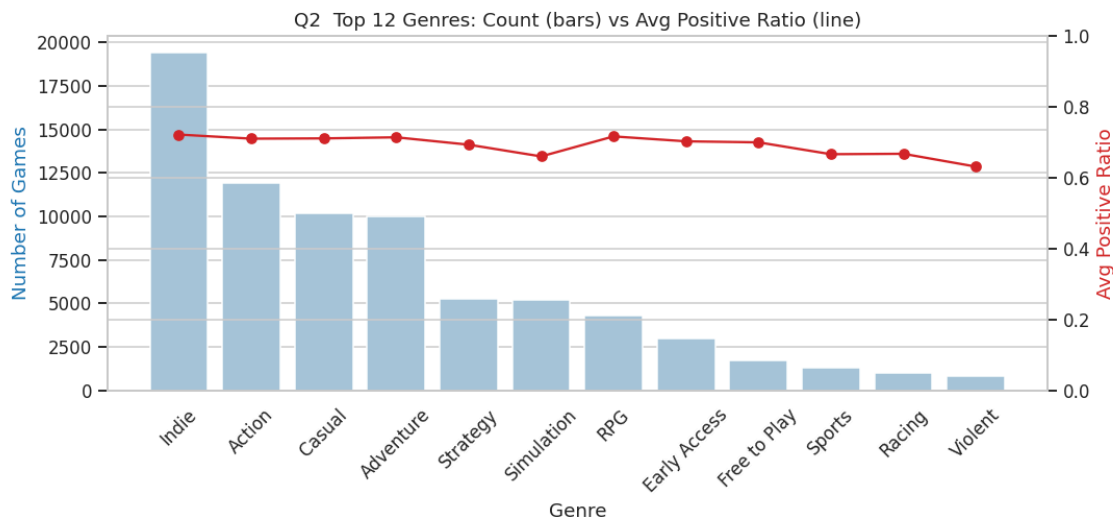
方法：按 release_year 分组计数（Spark SQL）。



发现：发行量在 2014 年后爆发式增长，**2018 年达峰值 8,159 款**；**93%（25,242 款）的游戏在 2014 年及以后发行**。这与 Steam 2017 年开放 Steam Direct、独立游戏大量涌入直接相关；2019 因数据为年中快照而回落。

Q2 热门类型及其口碑

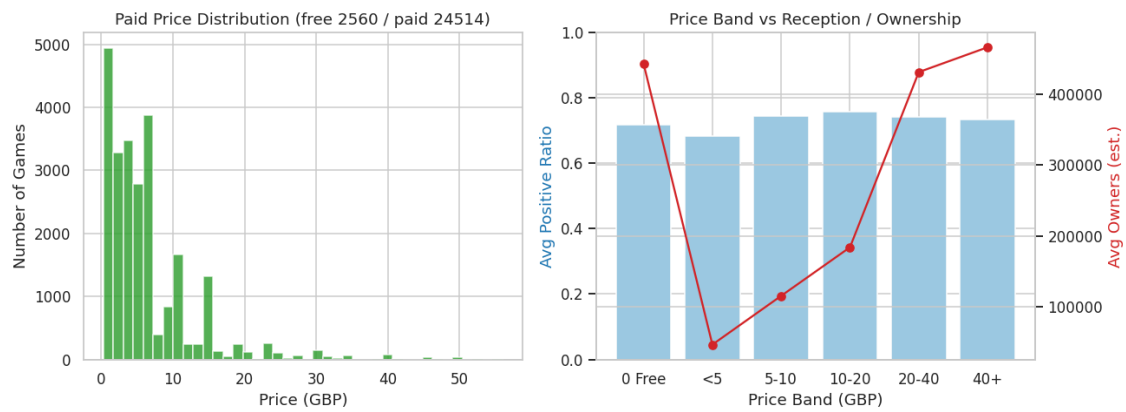
方法：explode(genres) 后按类型聚合数量与平均好评率。



发现：**Indie（19,419）、Action（11,903）、Casual（10,210）、Adventure（10,031）数量领先**。但数量多不等于口碑好——各类型平均好评率集中在 0.63–0.73，休闲/竞速/暴力类略低，核心向类型相对更稳。

Q3 价格 × 口碑 × 拥有量

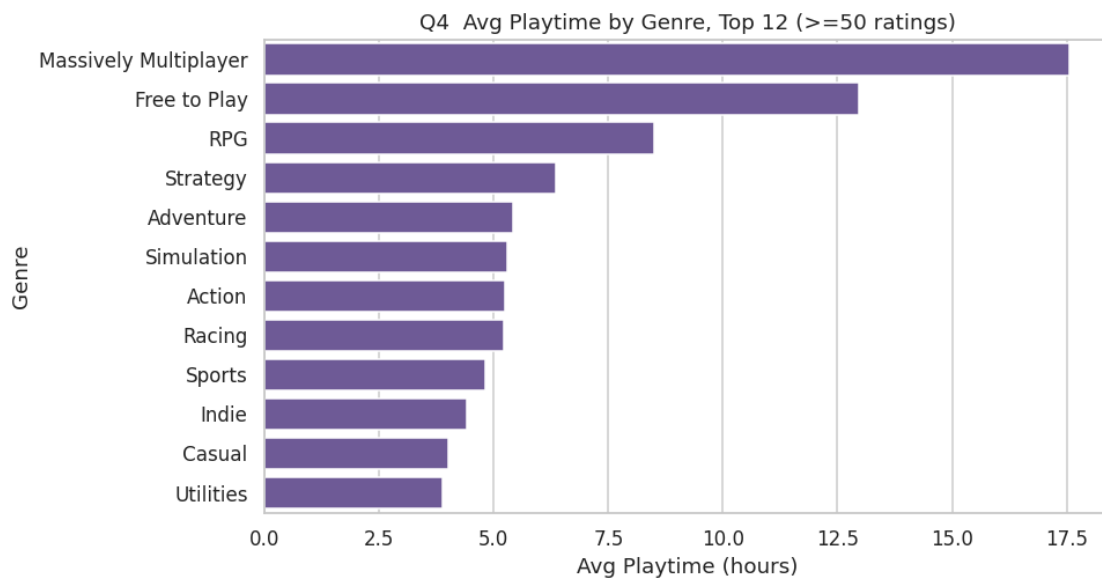
方法：价格分桶后聚合平均好评率与平均拥有量；并统计免费/付费占比。



发现：付费游戏中位价仅 **£3.99**，定价高度集中于低价区。关键反差：最低价区（<£5，13,285 款）平均好评率最低（**0.684**）、平均拥有量也最低（约 4.7 万），是典型的"走量但偏低质"长尾；而 **£10-40** 中端游戏好评率最高（约 0.75），免费游戏与中高价精品则拥有量最大（免费约 44 万）。即"低价不等于走量，免费与精品才走量"。

Q4 游玩时长 × 类型（用户粘性）

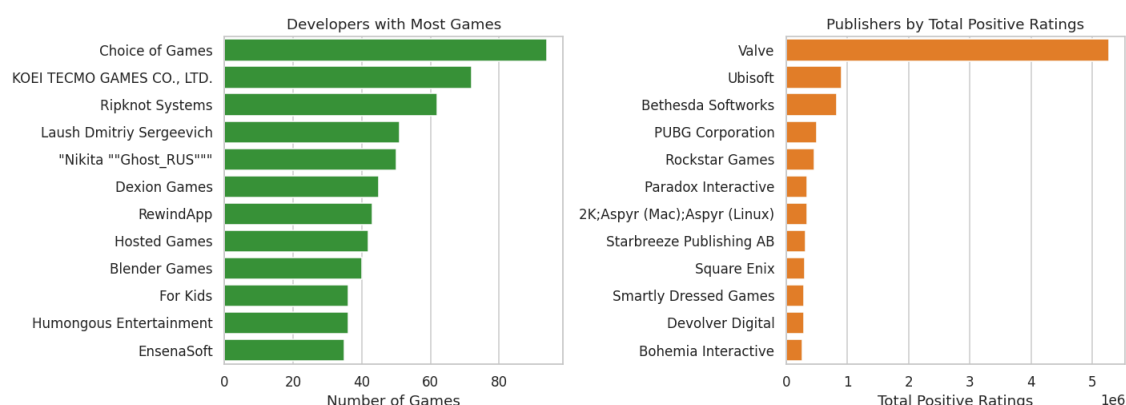
方法：在有一定评价量的样本上，按类型聚合平均游玩时长。



发现：大型多人（MMO，约 17.6 小时）、免费网游（13 小时）、RPG（8.5 小时）、策略（6.3 小时）粘性最强。强粘性高度依赖可重复游玩机制（多人对战、长期养成/经营），休闲解谜类时长最短。

Q5 Top 开发商 / 发行商

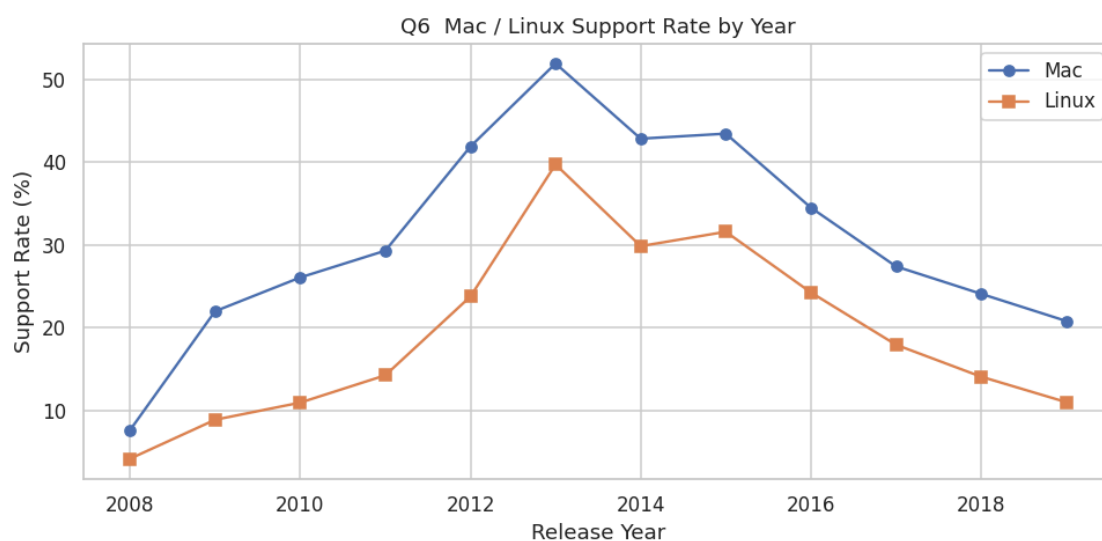
方法：按开发商计数；按发行商累计好评数排序（Spark SQL）。



发现：发行数量最多的是量产型工作室（Choice of Games 94 款、KOEI TECMO 72 款）；而累计口碑高度集中于头部发行商——**Valve（527 万好评）**一骑绝尘，其后为育碧、Bethesda、PUBG、Rockstar。数量与影响力并不等同：头部靠少数爆款贡献了绝大多数正面评价。

Q6 平台支持趋势

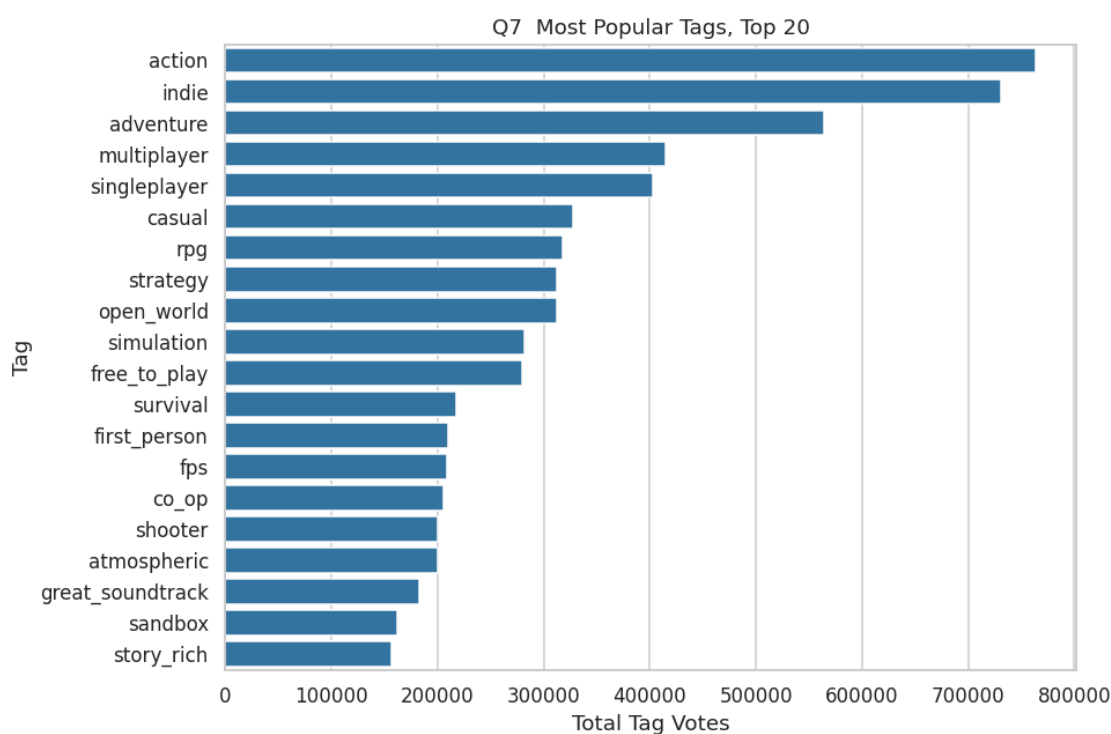
方法：统计整体平台占比，及 Mac/Linux 支持率随年份变化。



发现：Windows 支持率 **100%**，Mac **29.8%**、Linux **19.3%**。Mac/Linux 支持率在 **2013–2015 年（SteamOS / Steam Machine 推广期）**见顶，随该战略降温而回落——平台覆盖与厂商商业战略强相关。

Q7 最受欢迎标签

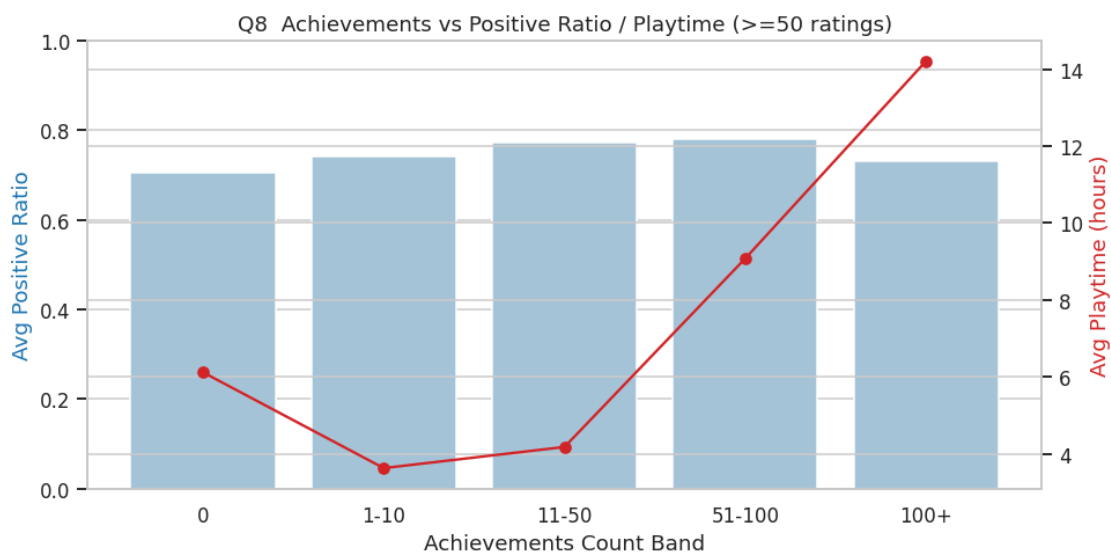
方法：在标签长表上按 tag 汇总投票数（Spark SQL）。



发现：投票最高的标签为 **action** (76.4 万)、**indie** (73.0 万)、**adventure**、**multiplayer**、**singleplayer**，与 Q2 的类型分布互相印证；玩家心智中的标签更偏"玩法体验"（如 singleplayer、atmospheric）。

Q8 成就数量 × 口碑 / 时长

方法：按成就数量分桶，比较平均好评率与平均时长。



发现：有成就的游戏好评率（0.74–0.78）整体高于无成就（0.704），并在 51–100 个成就时好评率最高（0.779）；游玩时长随成就数单调上升（100+ 成就达 14.2 小时）。说明成就作为正反馈/留存机制与满意度、粘性正相关，但数量过多边际收益递减。

5. 结论

1. **市场结构**：Steam 自 2014 年起进入独立游戏井喷期，Indie/Action/Casual 在数量上主导，但同质化也压低了长尾产品的口碑。
2. **商业规律**：定价集中于低价区，但**低价不等于走量**——<£5 多为低质长尾，免费与中高价精品才同时获得高拥有量；中端定价口碑最佳。
3. **用户粘性**：多人 / 网游 / RPG / 策略类时长最长，粘性源于可重复游玩与长期养成机制。
4. **平台与机制**：跨平台支持随厂商战略（SteamOS）起落；成就系统与玩家满意度、留存正相关。
5. **工程收获**：HDFS 原始区/清洗区分层、Parquet 分区、Spark SQL 聚合、Hive 外部表与预计算 Dashboard 串成了一个可复现的数据应用闭环，而不是只停留在离线图表分析。

6. 反思与局限

- 数据为 **2019 年快照**，无法反映近年趋势；owners 为区间估计、price 为英镑计价，分析结论偏宏观。
- 缺少**真实文本评论**，好评率只能由 positive/negative 计数近似，无法做情感细分。
- 未深入解析 description / requirements 中的文本与 HTML 字段。**后续可**：解析描述做 NLP 主题/情感分析、引入时间序列建模、用标签共现做社区/聚类分析。
- 工程上单机 HDFS 仅作教学演示，真实场景应扩展为多节点集群，并进一步补充调度、权限、元数据治理与质量监控。

7. 团队分工

成员	主要分工
罗景楠	数据工程与复现：HDFS 环境搭建、数据入库、Spark 清洗管线、Hive 数仓验证（setup/、src/clean_pipeline.py、src/hive_warehouse.py）
马亦麟	分析与展示：8 个分析问题的 Spark SQL、图表与解读、Streamlit 仪表盘（notebooks/、sql/、app/）
共同	报告撰写、结果校验、演示材料整理与 GitHub 交付

8. 评分细则对照

Rubric 要求	本项目对应交付
数据集介绍	第 1 节；dataset/README.md
HDFS 存储与访问策略	第 2 节；setup/；src/common.py
清洗、解析、缺失值处理	第 3 节；src/clean_pipeline.py
Spark / Hive / SQL 分析方法	第 4 节；notebooks/steam_analysis.ipynb；sql/analysis_queries.sql；sql/hive_setup.hql
关键发现与解释	第 4-5 节，8 个问题均给出数据结论
可视化与解读	figures/ 中 8 张图；第 2.5 节 Dashboard 截图；报告第 4 节；app/dashboard.py
结论与反思	第 5-6 节
完整可执行代码与扩展实现	src/*.py、notebooks/、sql/、app/；Streamlit Dashboard 作为加分项

附录：环境与复现

```
git clone https://github.com/yilinpotato/BigData-App.git
cd BigData-App
bash setup/00_install_stack.sh    # 安装 Hadoop (首次)
bash setup/01_start_hdfs.sh      # 启动单机 HDFS
bash setup/02_load_data.sh       # CSV 入 HDFS
source setup/env.sh
python src/clean_pipeline.py      # 清洗 -> Parquet
# 打开 notebooks/steam_analysis.ipynb 运行分析
python src/hive_warehouse.py     # 可选: 建立 Hive 外部表并执行 HiveQL 查询
python src/export_dashboard_data.py
streamlit run app/dashboard.py    # 可选: 启动交互式仪表盘
```